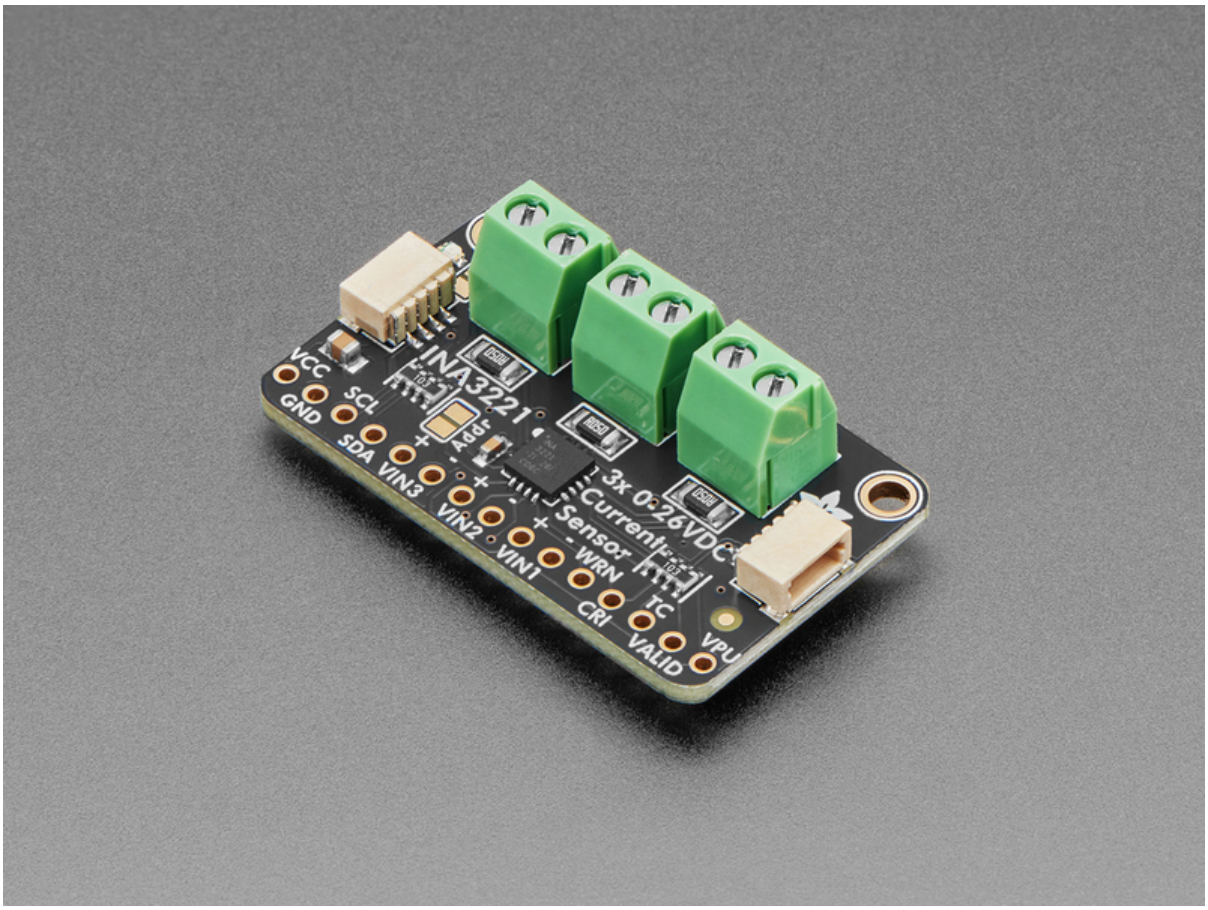
Adafruit INA3221 Breakout

Created by Liz Clark

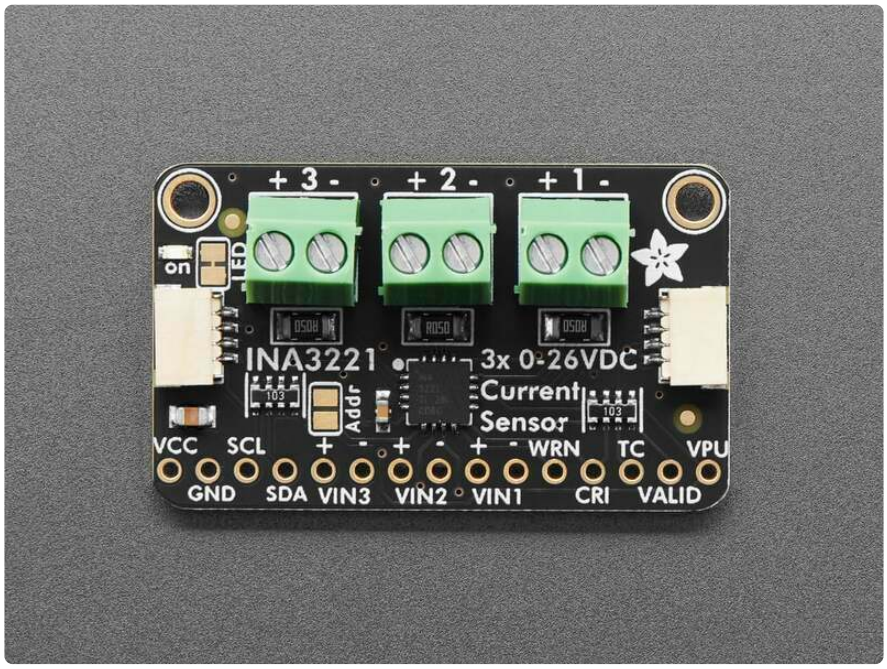


https://learn.adafruit.com/adafruit-ina3221-breakout

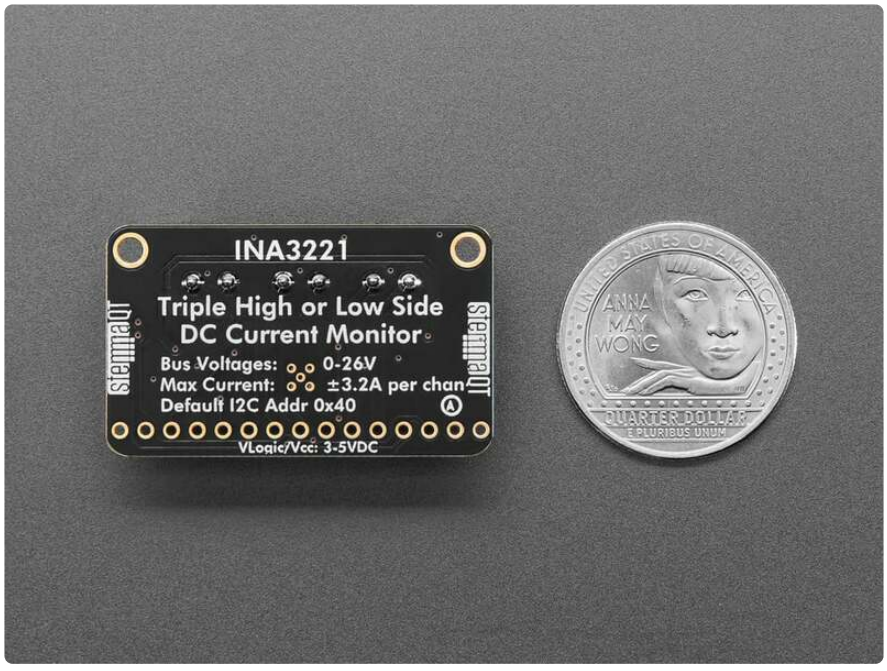
Last updated on 2025-08-15 01:50:04 PM EDT

Table of Contents

Overview

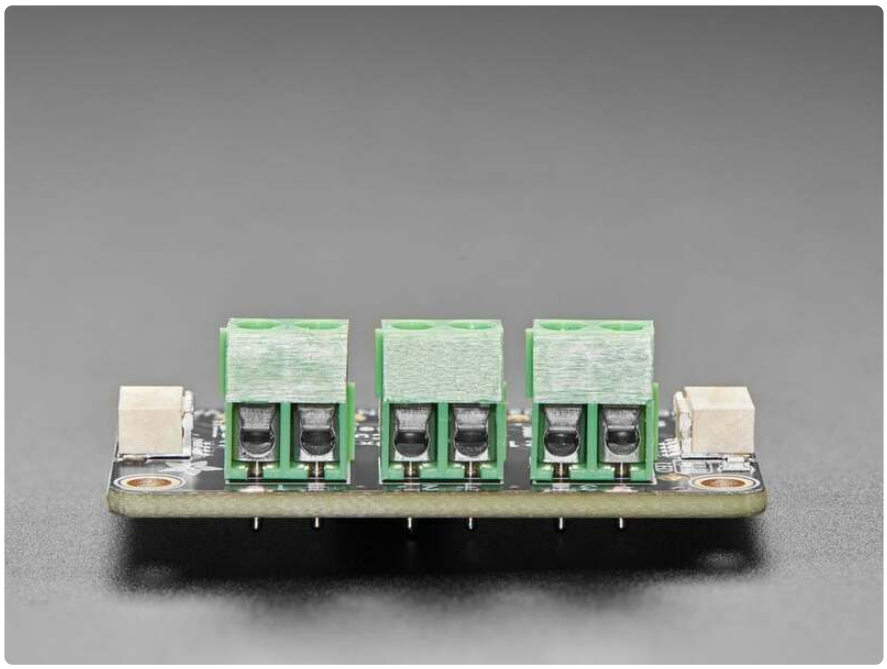


This breakout board will solve all your multi-rail power-monitoring problems. Instead of struggling with up to 6 multimeters, you can just use the handy INA3221 chip on this breakout to both measure both the high side voltage and DC current draw of up to three power supplies over I2C with ±1% precision.

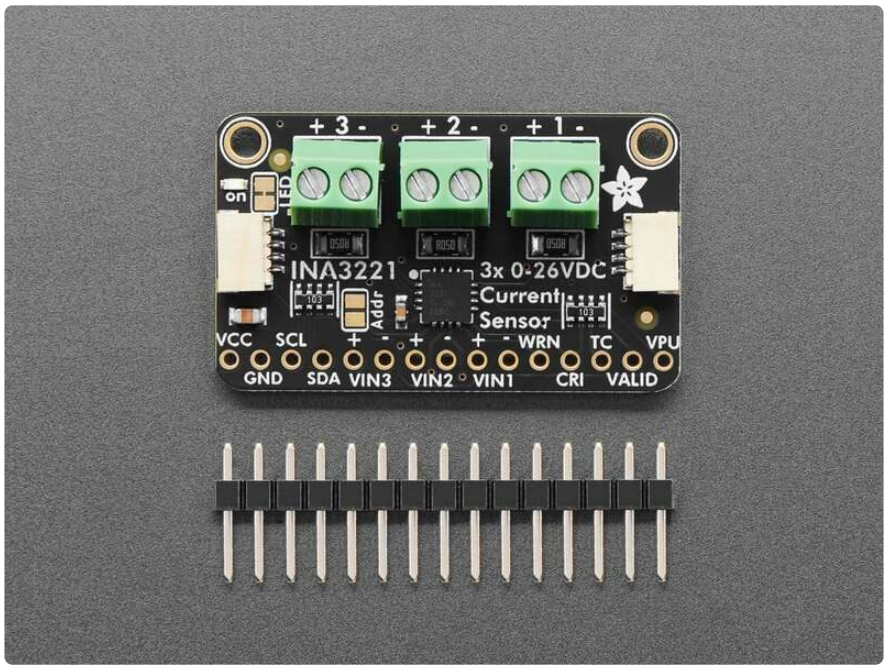


Most current-measuring devices such as our current panel meter are only good for low side measuring. That means that unless you want to get a battery involved, you have to stick the measurement resistor between the target ground and true ground. This can cause problems with circuits since electronics tend to not like it when the

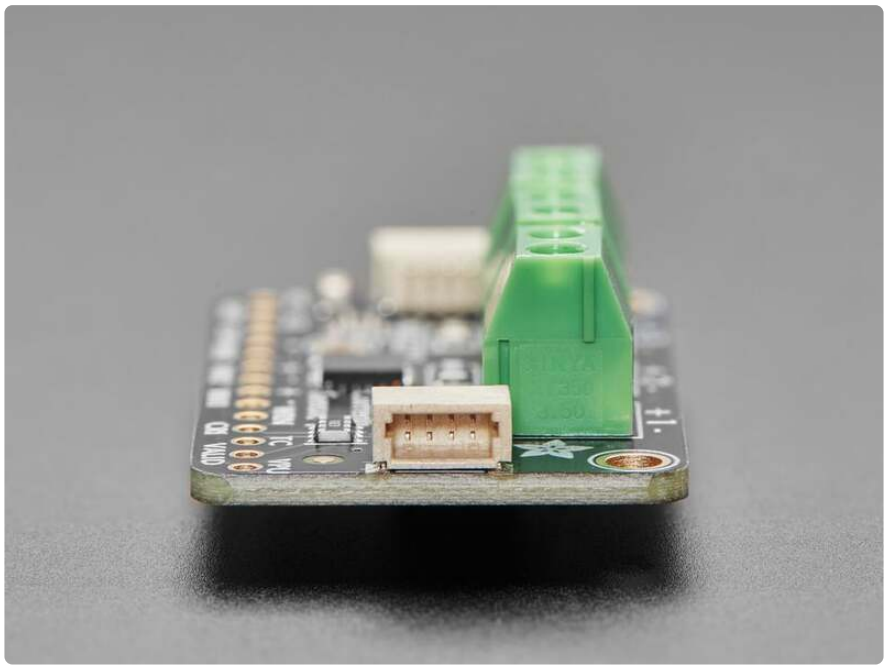
ground references change and move with varying current draw. This chip is much smarter - it can handle high or low side current measuring, up to +26VDC, even though it is powered with 3 or 5V. It will also report back that high side voltage, which is great for tracking battery life or solar panels.



A precision amplifier measures the voltage across the built-in 0.05 ohm, 1% sense resistors. Since the amplifier maximum input difference is ±163.8mV this means it can measure up to ±3.2 Amps. With the internal 13 bit ADC, the resolution at ±3.2A range is 0.4mA. Advanced hackers can remove the 0.05 ohm current sense resistor and replace it with their own to change the range (say a 0.01 ohm to measure up 16.4 Amps with a resolution of 2mA)

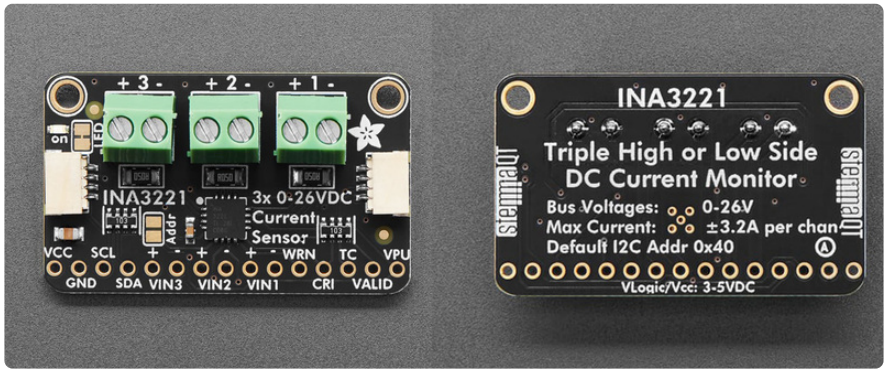


Usage is simple. Power the sensor itself with 3 to 5VDC and connect the two I2C pins up to your microcontroller - the logic level will be the same as the Vin power level. Then connect your target power supplies to VIN1/2/3+ and the loads to ground to VIN1/2/3-. If you want, set up over-current alerts on the warning or critical IRQ pins. We have libraries in both Arduino and CircuitPython/Python so you can use it with boards as simple as Arduino-compatible ATmegas all the way up to the latest Raspberry Pi's.



You **don't even need to solder the I2C and power lines**: we pre-attach 3.5mm terminal blocks and added SparkFun qwiic (https://adafru.it/Fpw) compatible STEMMA QT **(https://adafru.it/Ft4)** connectors for the I2C bus. Just wire up to your favorite micro using a STEMMA QT adapter cable. (https://adafru.it/JnB) The Stemma QT connectors also mean the INA3221 can be used with our various associated accessories. (https://adafru.it/Ft6) QT Cable is not included, but we have a variety in the shop (https://adafru.it/17VE). If you need access to all of the pins, we include 0.1" pin header (so you can easily attach this sensor to a breadboard).

Pinouts



Default I2C address is **0x40**.

Power Pins

- **VCC** - this is the power pin. To power the board, give it the same power as the logic level of your microcontroller - e.g. for a 5V micro like Arduino, use 5V. It can be powered between from 2.7 V to 5.5 V.
- **GND** - common ground for power and logic.
- **VPU** - this is the pull-up supply voltage pin. It is used to provide bias to the power valid (**VALID**) interrupt pin. Connect this pin to **VCC** to match the same logic level of your microcontroller.

I2C Logic Pins

- **SCL** - I2C clock pin, connect to your microcontroller's I2C clock line. This pin can use 3-5V logic, and there's a **10K pullup** on this pin.
- **SDA** - I2C data pin, connect to your microcontroller's I2C data line. This pin can use 3-5V logic, and there's a **10K pullup** on this pin.
- STEMMA QT **(https://adafru.it/Ft4) -** These connectors allow you to connect to development boards with **STEMMA QT** (Qwiic) connectors or to other things with various associated accessories (https://adafru.it/Ft6).

Input Channels

There are three input channels available on the INA3221: **VIN1**, **VIN2** and **VIN3**. Each input has a positive (+) and a negative (-) line. You'll connect your target power supplies to the positive line (**VIN1/2/3+**) and the loads to the negative ground line (**VIN 1/2/3-**). These inputs are available as pins along the bottom of the board and via terminal blocks at the top; no soldering required.

You can measure high or low side current measuring up to +26VDC. The breakout will also report back the high side voltage, which is great for tracking battery life or solar panels. A precision amplifier measures the voltage across the built-in 0.05 ohm, 1% sense resistors. Since the amplifier maximum input difference is ±163.8mV, this means it can measure up to ±3.2 Amps. With the internal 13 bit ADC, the resolution at ±3.2A range is 0.4mA.

Interrupts

- **WRN** - This is the averaged measurement warning alert interrupt pin. This interrupt is triggered if the averaged current reading on any input channel exceeds a set threshold. If the interrupt is triggered, the **WRN** pin goes low.
- **CRI** - This is the conversion-triggered critical alert interrupt pin. It detects overcurrent on any of the input channels. If the interrupt is triggered, the **CRI** pin goes low.

- **TC** - This is the timing control alert interrupt pin. It signals if the power supplies connected to the input channels do not power up in the correct sequence. If one bus voltage doesn't reach its target level within a set time after another, the **TC** pin goes low.
- **VALID** - This is the power valid alert interrupt pin. You can set an upper and lower limit threshold for this interrupt. When the upper limit interrupt is triggered, the pin goes high. When the lower limit interrupt is triggered, the pin goes low. You will need to connect the **VPU** pin to the same voltage as your microcontroller logic level (3V or 5V) to use the **VALID** interrupt.

Address Jumper

- **Addr** - On the front of the board, below the **INA3221 label** on the board silk, is the I2C address jumper. It is labeled **Addr** on the board silk. You can solder this jumper closed to change the I2C address to **0x41**. Otherwise, the I2C address is **0x40** when the jumper is open (default).

Power LED and Jumper

- **Power LED** - on the left side of the board, above the STEMMA QT connector, is the power LED, labeled **on**. It is a green LED.
- **LED jumper** - on the front of the board, directly to the right of the power LED, is a jumper for the power LED. It is labeled **LED** on the board silk. If you want to disable the power LED, cut the trace on this jumper.

CircuitPython and Python

It's easy to use the **INA3221** with Python or CircuitPython, and the Adafruit_CircuitPython_INA3221 (https://adafru.it/1a9m) module. This module allows you to easily write Python code to read voltage and current draw from the three channels on the breakout.

You can use this driver with any CircuitPython microcontroller board or with a computer that has GPIO and Python thanks to Adafruit_Blinka, our CircuitPython-for-Python compatibility library (https://adafru.it/BSN).

You may find that using DC jacks with terminal blocks to interface between your power supply and load (device) will make interfacing with the INA3221 a little easier.

Female DC Power adapter - 2.1mm jack to screw terminal block

If you need to connect a DC power wall wart to a board that doesn't have a DC jack - this adapter will come in very handy! There is a 2.1mm DC jack on one end, and a screw terminal...

https://www.adafruit.com/product/368

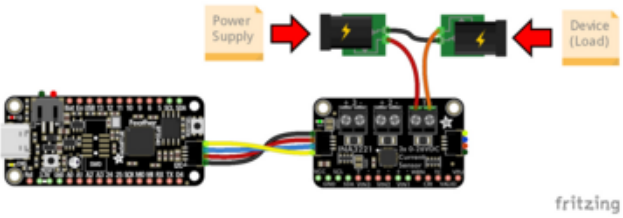
Male DC Power adapter - 2.1mm plug to screw terminal block

If you need to connect a battery pack or wired power supply to a board that has a DC jack - this adapter will come in very handy! There is a 2.1mm DC plug on one end, and a screw...

https://www.adafruit.com/product/369

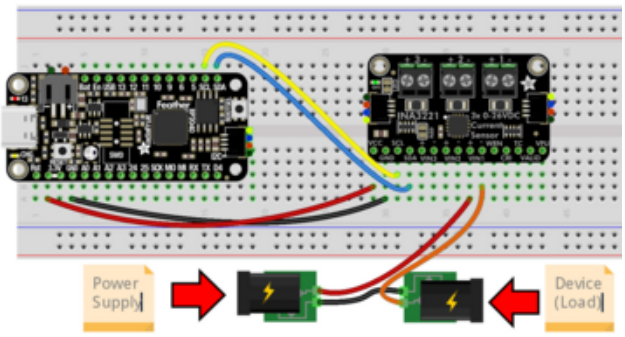
CircuitPython Microcontroller Wiring

First wire up the breakout to your board exactly as follows. The following is the breakout wired to a Feather RP2040 using the STEMMA connector with a power supply and device (load) connected to channel 1. You can use the same power supply and load wiring for the additional two channels (power supply to + and load to -):



Board STEMMA 3V to **breakout VIN (red wire)**
Board STEMMA GND to **breakout GND (black wire)**
Board STEMMA SCL to **breakout SCL (yellow wire)**
Board STEMMA SDA to **breakout SDA (blue wire)**
Power supply GND to **load GND (black wire)**
Power supply positive to **breakout 1+ (red wire)**
Load positive to **breakout 1- (orange wire)**

The following is the breakout wired to a Feather RP2040 using a solderless breadboard:

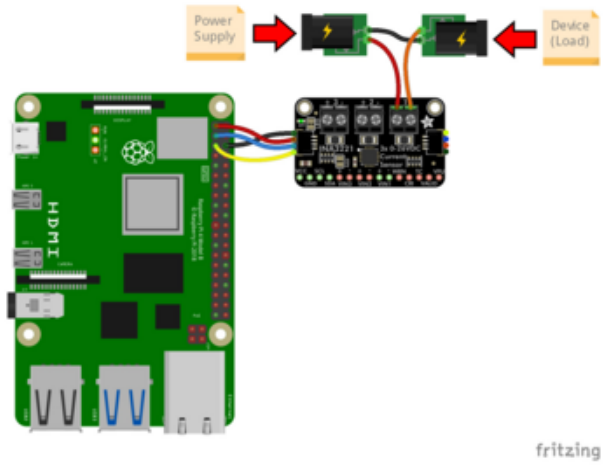


Board 3V to **breakout VIN (red wire)**
Board GND to **breakout GND (black wire)**
Board SCL to **breakout SCL (yellow wire)**
Board SDA to **breakout SDA (blue wire)**
Power supply GND to **load GND (black wire)**
Power supply positive to **breakout 1+ (red wire)**
Load positive to **breakout 1- (orange wire)**

Python Computer Wiring

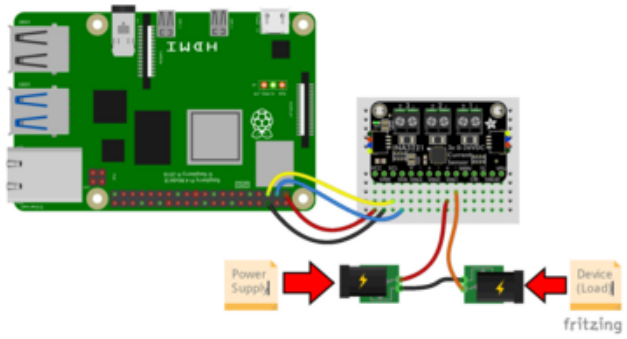
Since there are dozens of Linux computers/boards you can use, we will show wiring for Raspberry Pi. For other platforms, [please visit the guide for CircuitPython on Linux to see whether your platform is supported](https://adafru.it/BSN) (https://adafru.it/BSN).

Here's the Raspberry Pi wired with I2C using the STEMMA connector with a power supply and device (load) connected to channel 1. You can use the same power supply and load wiring for the additional two channels (power supply to + and load to -):



Pi 3V to **breakout VIN (red wire)**
Pi GND to **breakout GND (black wire)**
Pi SCL to **breakout SCL (yellow wire)**
Pi SDA to **breakout SDA (blue wire)**
Power supply GND to **load GND (black wire)**
Power supply positive to **breakout 1+ (red wire)**
Load positive to **breakout 1- (orange wire)**

Here's the Raspberry Pi wired with I2C using a solderless breadboard:



Pi 3V to **breakout VIN (red wire)**
Pi GND to **breakout GND (black wire)**
Pi SCL to **breakout SCL (yellow wire)**
Pi SDA to **breakout SDA (blue wire)**
Power supply GND to **load GND (black wire)**
Power supply positive to **breakout 1+ (red wire)**
Load positive to **breakout 1- (orange wire)**

Python Installation of INA3221 Library

You'll need to install the **Adafruit_Blinka** library that provides the CircuitPython support in Python. This may also require enabling I2C on your platform and verifying you are running Python 3. Since each platform is a little different, and Linux changes often, please visit the CircuitPython on Linux guide to get your computer ready (https://adafru.it/BSN)!

Once that's done, from your command line run the following command:

- `pip3 install adafruit-circuitpython-ina3221`

If your default Python is version 3 you may need to run 'pip' instead. Just make sure you aren't trying to use CircuitPython on Python 2.x, it isn't supported!

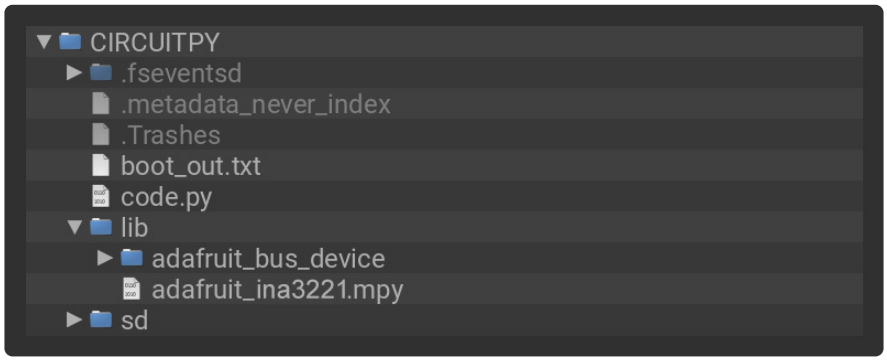
CircuitPython Usage

To use with CircuitPython, you need to first install the **Adafruit_CircuitPython_INA3221** library, and its dependencies, into the **lib** folder on your **CIRCUITPY** drive. Then you need to update **code.py** with the example script.

Thankfully, we can do this in one go. In the example below, click the **Download Project Bundle** button below to download the necessary libraries and the **code.py** file in a zip file. Extract the contents of the zip file, and copy the **entire lib folder** and the **code.py** file to your **CIRCUITPY** drive.

Your **CIRCUITPY/lib** folder should contain the following folder and file:

- **adafruit_bus_device/**
- **adafruit_ina3221.mpy**



Python Usage

Once you have the library `pip3` installed on your computer, copy or download the following example to your computer, and run the following, replacing **code.py** with whatever you named the file:

```
python3 code.py
```

Example Code

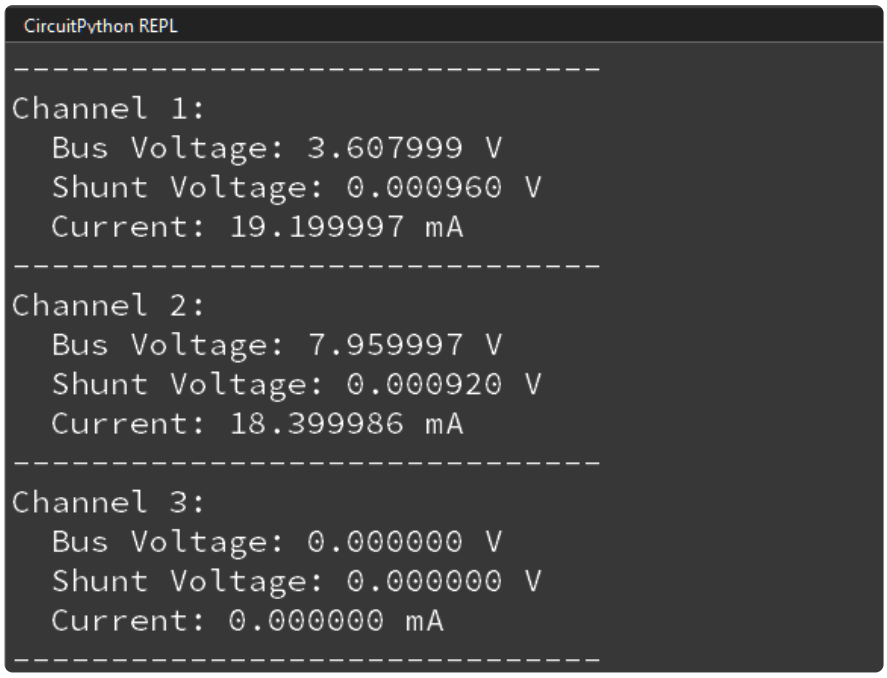
If running CircuitPython: Once everything is saved to the **CIRCUITPY** drive, connect to the serial console (https://adafru.it/Bec) to see the data printed out!

If running Python: The console output will appear wherever you are running Python.

```
# SPDX-FileCopyrightText: Copyright (c) 2024 Liz Clark for Adafruit Industries
#
# SPDX-License-Identifier: MIT
import time

import board

from adafruit_ina3221 import INA3221
```

```
i2c = board.I2C()
ina = INA3221(i2c, enable=[0, 1, 2])

while True:
    for i in range(3):
        bus_voltage = ina[i].bus_voltage
        shunt_voltage = ina[i].shunt_voltage
        current = ina[i].current

        print(f"Channel {i + 1}:")
        print(f"  Bus Voltage: {bus_voltage:.6f} V")
        print(f"  Shunt Voltage: {shunt_voltage:.6f} mV")
        print(f"  Current: {current:.6f} mA")
        print("-" * 30)

    time.sleep(2)
```

The INA3221 is instantiated over I2C. Then in the loop, the voltage and current readings for all three channels are printed to the serial monitor every two seconds. In the screenshot below, a QT Py CH32V203 running a NeoPixel swirl example was connected to channel 1 and a 9V guitar pedal was connected to channel 2. Nothing was connected to channel 3.



Python Docs

Python Docs (https://adafru.it/1a9k)

Arduino

Using the INA3221 breakout with Arduino involves wiring up the breakout to your Arduino-compatible microcontroller, installing the Adafruit_INA3221 (https://adafru.it/1a9n) library, and running the provided example code.

You may find that using DC jacks with terminal blocks to interface between your power supply and load (device) will make interfacing with the INA3221 a little easier.

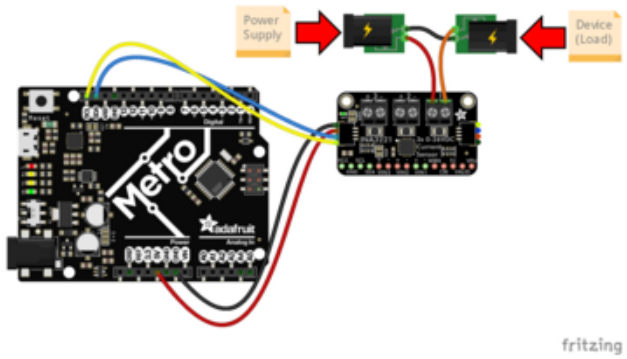
Female DC Power adapter - 2.1mm jack to screw terminal block
If you need to connect a DC power wall wart to a board that doesn't have a DC jack - this adapter will come in very handy! There is a 2.1mm DC jack on one end, and a screw terminal...
https://www.adafruit.com/product/368

Male DC Power adapter - 2.1mm plug to screw terminal block
If you need to connect a battery pack or wired power supply to a board that has a DC jack - this adapter will come in very handy! There is a 2.1mm DC plug on one end, and a screw...
https://www.adafruit.com/product/369

Wiring

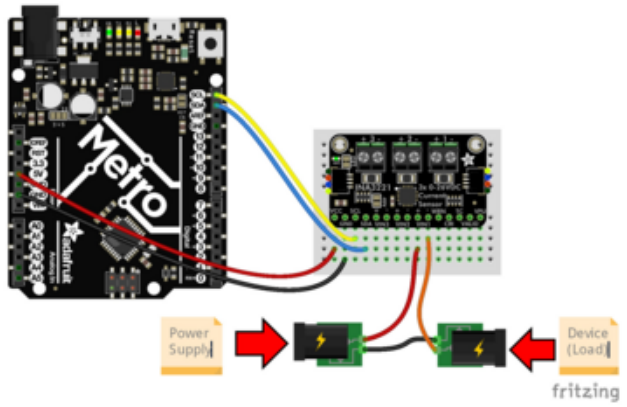
Wire as shown for a **5V** board like an Uno. If you are using a **3V** board, like an Adafruit Feather, wire the board's 3V pin to the breakout VIN.

Here is an Adafruit Metro wired up to the breakout using the STEMMA QT connector with a power supply and device (load) connected to channel 1. You can use the same power supply and load wiring for the additional two channels (power supply to + and load to -):



Board 5V to **breakout VIN (red wire)**
Board GND to **breakout GND (black wire)**
Board SCL to **breakout SCL (yellow wire)**
Board SDA to **breakout SDA (blue wire)**
Power supply GND to **load GND (black wire)**
Power supply positive to **breakout 1+ (red wire)**
Load positive to **breakout 1- (orange wire)**

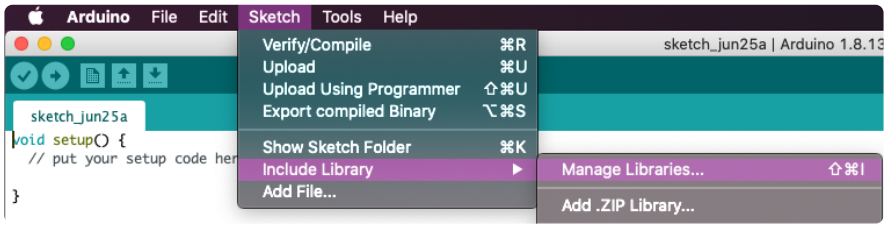
Here is an Adafruit Metro wired up using a solderless breadboard:



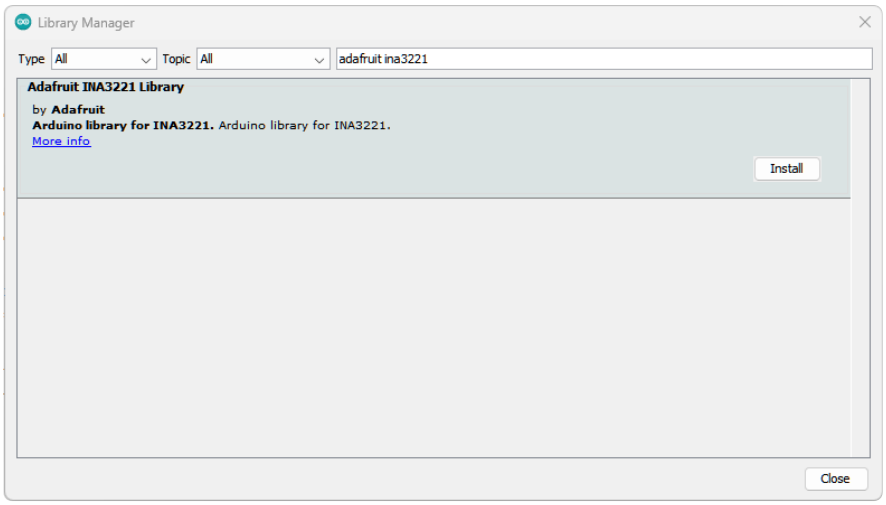
Board 5V to **breakout VIN (red wire)**
Board GND to **breakout GND (black wire)**
Board SCL to **breakout SCL (yellow wire)**
Board SDA to **breakout SDA (blue wire)**
Power supply GND to **load GND (black wire)**
Power supply positive to **breakout 1+ (red wire)**
Load positive to **breakout 1- (orange wire)**

Library Installation

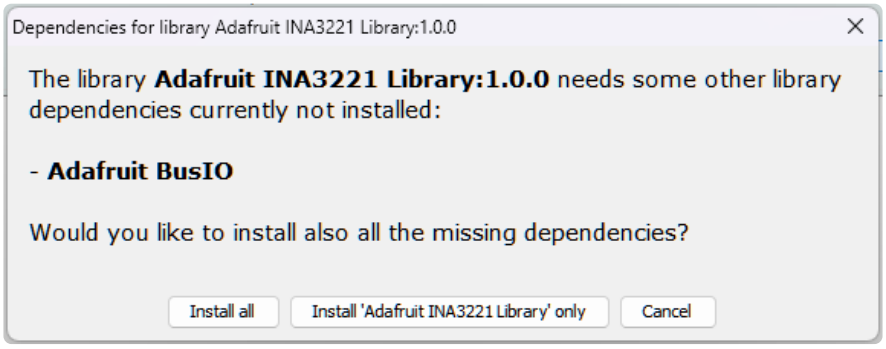
You can install the **Adafruit_INA3221** library for Arduino using the Library Manager in the Arduino IDE.



Click the **Manage Libraries ...** menu item, search for **Adafruit_INA3221**, and select the **Adafruit INA3221** library:



If asked about dependencies, click "Install all".



If the "Dependencies" window does not come up, then you already have the dependencies installed.

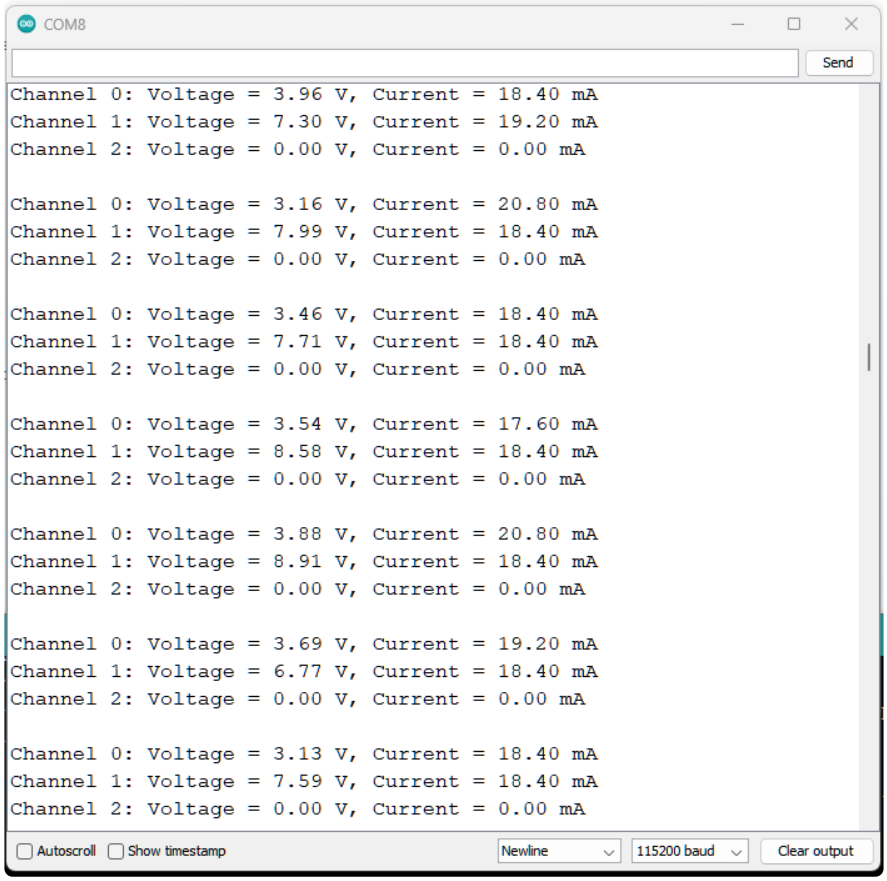
If the dependencies are already installed, make sure you update them through the Arduino Library Manager before loading the example!

Example Code

```
#include "Adafruit_INA3221.h"
#include <Wire.h>

// Create an INA3221 object
Adafruit_INA3221 ina3221;

void setup() {
  Serial.begin(115200);
  while (!Serial)
    delay(10); // Wait for serial port to connect on some boards

  Serial.println("Adafruit INA3221 simple test");

  // Initialize the INA3221
  if (!ina3221.begin(0x40, &Wire)) { // can use other I2C addresses or buses
    Serial.println("Failed to find INA3221 chip");
```

```
    while (1)
      delay(10);
  }
  Serial.println("INA3221 Found!");

  ina3221.setAveragingMode(INA3221_AVG_16_SAMPLES);

  // Set shunt resistances for all channels to 0.05 ohms
  for (uint8_t i = 0; i < 3; i++) {
    ina3221.setShuntResistance(i, 0.05);
  }

  // Set a power valid alert to tell us if ALL channels are between the two
  // limits:
  ina3221.setPowerValidLimits(3.0 /* lower limit */, 15.0 /* upper limit */);
}

void loop() {
  // Display voltage and current (in mA) for all three channels
  for (uint8_t i = 0; i < 3; i++) {
    float voltage = ina3221.getBusVoltage(i);
    float current = ina3221.getCurrentAmps(i) * 1000; // Convert to mA

    Serial.print("Channel ");
    Serial.print(i);
    Serial.print(": Voltage = ");
    Serial.print(voltage, 2);
    Serial.print(" V, Current = ");
    Serial.print(current, 2);
    Serial.println(" mA");
  }

  Serial.println();
  // Delay for 250ms before the next reading
  delay(250);
}
```

Upload the sketch to your board and open up the Serial Monitor (**Tools -> Serial Monitor**) at 115200 baud. After the INA3221 is recognized over I2C, you'll see the voltage and current readings for all three channels print to the Serial Monitor every 250 milliseconds. In the screenshot below, a QT Py CH32V203 running a NeoPixel swirl example was connected to channel 0 and a 9V guitar pedal was connected to channel 1. Nothing was connected to channel 2.



Arduino Docs

Arduino Docs (https://adafru.it/1a9l)

Downloads

Files

- INA3221 Datasheet (https://adafru.it/1a9o)
- EagleCAD PCB Files on GitHub (https://adafru.it/1a9p)
- 3D models on GitHub (https://adafru.it/1apQ)
- Fritzing object in the Adafruit Fritzing Library (https://adafru.it/1a9q)

Schematic and Fab Print